ASSIGNMENT 01

How to Write a Device Driver

Course: Embedded Linux Systems
Student: Nguyen Thanh Phong
Student ID: SE182514

---

1. Introduction

This assignment explores Linux kernel device driver development using an ARMv7 environment running under QEMU.

Two kernel modules were developed and tested:

1. IOCTL Character Device Driver (`asgn1_driver`)
2. Atomic Counter Driver (`counter_driver`)

The implementation demonstrates communication between user space and kernel space, device registration, synchronization, ioctl commands, and the procfs interface.

2. Development Environment

- Host operating system: Ubuntu Linux
- Target architecture: ARMv7
- Linux kernel: 5.15
- Emulator: QEMU (vexpress-a9)
- Cross-compiler: arm-linux-gnueabihf-gcc
- Root filesystem: BusyBox-based initramfs

3. Task 1 – IOCTL Character Device Driver

3.1. Driver Architecture

The asgn1_driver is a Linux character device driver implemented as a loadable kernel module. It provides the device node `/dev/asgn1` and uses major number 242.

The driver communicates with user-space applications through standard file operations and ioctl commands.

The main communication flow is:

User Application -> /dev/asgn1 -> VFS -> File Operations -> Kernel Driver -> Internal Buffer

3.2. IOCTL Commands

Four ioctl commands are implemented:

Command	Description
RESET_BUFFER	Clears the buffer and resets read/write statistics
GET_STATS	Returns driver statistics to user space
SET_MODE	Selects Kernel Mode or Echo Mode
GET_VERSION	Returns the driver version string

In Kernel Mode, the driver adds the prefix `[KERNEL]` to written data. In Echo Mode, data is stored without the prefix.

3.3. Synchronization

The driver uses a mutex to protect shared state accessed by read, write, and ioctl operations.

This helps prevent concurrent operations from corrupting the internal buffer or producing inconsistent statistics.

3.4. Testing Results

The driver was tested using QEMU ARMv7 with Linux kernel 5.15.

Observed results included:

- Successful module loading and unloading.
- Reading and writing through `/dev/asgn1`.
- Correct `[KERNEL]Hello` output in Kernel Mode.
- Successful buffer reset and statistics reset.
- Correct `ENOTTY` response for an unsupported ioctl command.

4. Task 2 – Atomic Counter Driver

4.1. Driver Architecture

The counter_driver is an independent Linux kernel module that registers a miscellaneous character device named `/dev/counter`.

Unlike the first driver, it uses the miscdevice framework, which automatically assigns a dynamic minor number under major number 10.

The driver also creates `/proc/counter_info` to provide information about its internal state.

4.2. Atomic Counter

The driver uses three atomic variables:

- `counter`: Stores the current counter value.
- `read_count`: Tracks successful read operations.
- `write_count`: Tracks successful write operations.

The `atomic_fetch_inc()` operation retrieves the current counter value and increments it atomically.

For example, three consecutive reads return:

0 -> 1 -> 2

After these reads, the internal counter value becomes 3.

4.3. Read and Write Operations

The read operation returns the current counter value as an ASCII string and then increments the counter.

The file offset is checked to ensure that each open file reaches EOF after returning one counter value.

The write operation uses `copy_from_user()` and `kstrtoint()` to convert user input into an integer.

Only non-negative integers are accepted. Invalid inputs return an error without changing the counter.

4.4. Procfs Interface

The `/proc/counter_info` interface displays:

- Current counter value
- Number of successful read operations
- Number of successful write operations

The interface is implemented using `seq_file` and `proc_ops`.

4.5. Testing Results

The counter driver was tested in QEMU ARMv7.

Observed results:

- The driver registered `/dev/counter` using major number 10.
- Three consecutive reads returned 0, 1, and 2.
- Writing 100 updated the counter successfully.
- Reading after the write returned 100.
- Negative values and text input were rejected.
- The procfs interface displayed the expected statistics.
- The driver was successfully unloaded using `rmmod`.

5. Technical Discussion

5.1. Character, Block, and Network Drivers

Linux supports different driver categories depending on how hardware and software exchange data.

Feature	Character Driver	Block Driver	Network Driver
Data model	Byte stream	Fixed-size blocks	Network packets
Typical interface	`/dev` + read/write	`/dev` + block I/O	Network interfaces
Examples	Serial port, sensor	SSD, SD card	Ethernet, Wi-Fi
Main purpose	Sequential data access	Storage access	Packet transmission

Both drivers in this assignment are character drivers because they exchange small amounts of data using file operations.

5.2. register_chrdev() vs miscdevice

Task 1 uses a character device with major number 242, whereas Task 2 uses the miscdevice framework.

The `register_chrdev()` interface requires the driver to manage the character device registration and device-node integration.

In contrast, `misc_register()` provides a simpler registration mechanism. Miscellaneous devices share major number 10 and receive individual minor numbers.

The miscdevice approach is suitable for small drivers such as the atomic counter.

5.3. Mutex vs Atomic Operations

A mutex protects a critical section containing several operations that must be performed together.

Task 1 uses a mutex when modifying its shared data buffer, operating mode, and associated statistics.

An atomic operation protects a single variable update without requiring an explicit mutex.

Task 2 uses `atomic_t` for its counter and read/write statistics.

Atomic variables are appropriate for individual counter updates, but they do not automatically make multiple related operations one indivisible transaction.

5.4. IOCTL Design

The ioctl interface allows applications to perform control operations that are not naturally represented by read() or write().

The driver defines four commands using Linux ioctl macros: `_IO`, `_IOR`, and `_IOW`.

A dedicated magic value helps distinguish the driver's ioctl command family.

For user-space data exchange, the driver uses `copy_to_user()` and `copy_from_user()`.

Unsupported ioctl requests return `-ENOTTY`.

6. Testing Results

6.1. Task 1 – IOCTL Driver Testing

The driver was tested on an ARMv7 Linux 5.15 system using QEMU.

The following tests were performed:

Test	Observed Result	Status
Module loading	Loaded with major 242	PASS
Character device	`/dev/asgn1` available	PASS
Read/write	Successfully echoed test data	PASS
GET_VERSION	Returned driver version	PASS
GET_STATS	Returned driver statistics	PASS
SET_MODE	Kernel mode produced `[KERNEL]Hello`	PASS
RESET_BUFFER	Buffer and read/write counts reset	PASS
Invalid ioctl	Returned ENOTTY	PASS
Module unloading	Successfully unloaded	PASS

Figure 1. Task 1 character device read/write test.

Figure 2. Task 1 ioctl command testing.

Figure 3. Task 1 procfs interface and module unloading.

6.2. Task 2 – Atomic Counter Driver Testing

The counter driver was tested using the same QEMU environment.

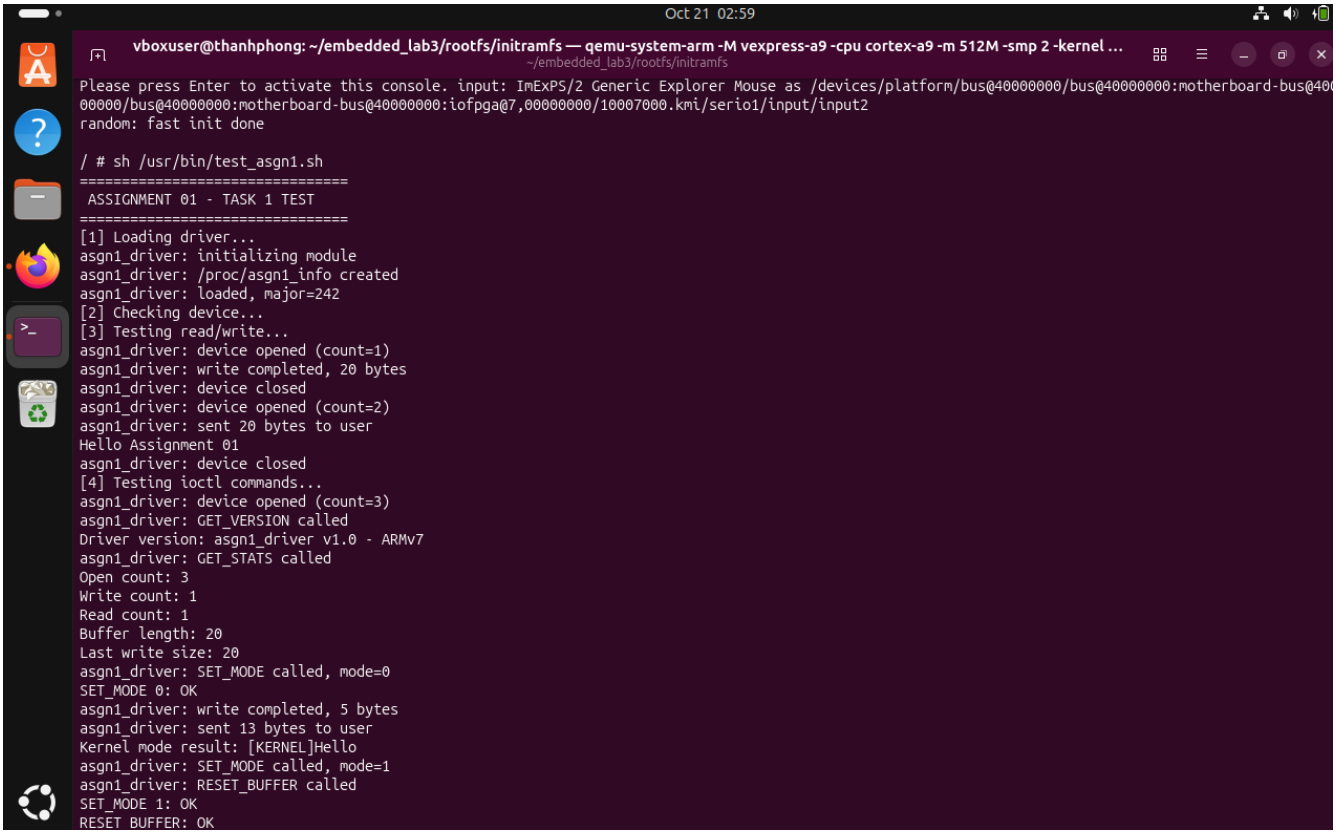


Figure 1: Task 1 - Read and write

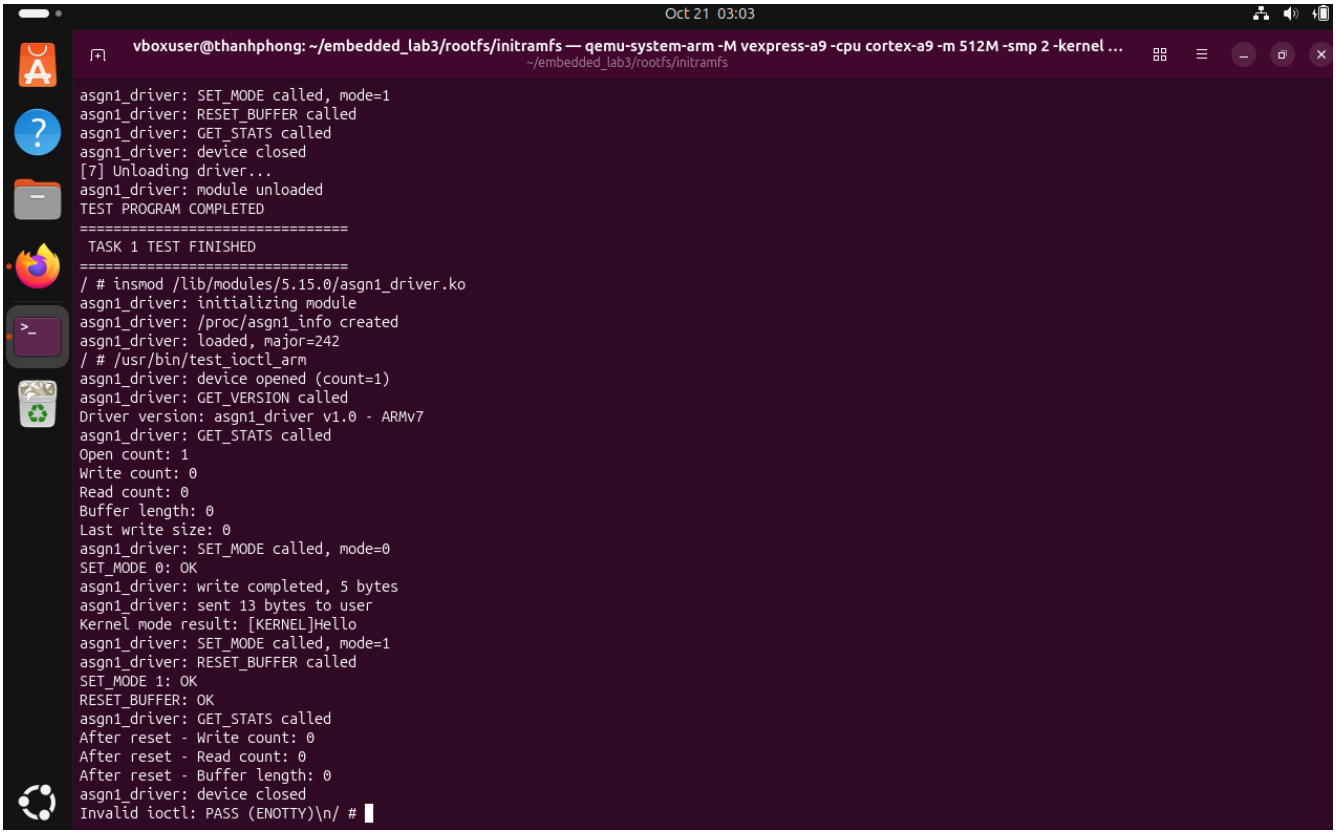


Figure 2: Task 1 - IOCTL tests



Figure 3: Task 1 - Procfs and module unloading

Test	Observed Result	Status
Module loading	Registered miscellaneous device, major 10	PASS
Initial counter	Started at 0	PASS
Repeated reads	Returned 0, 1, and 2	PASS
Writing an integer	Successfully accepted 100	PASS
Reading after write	Returned 100	PASS
Negative input	Rejected -5	PASS
Non-numeric input	Rejected abc	PASS
Procfs statistics	Displayed counter, reads, and writes	PASS
Module unloading	Successfully unloaded	PASS

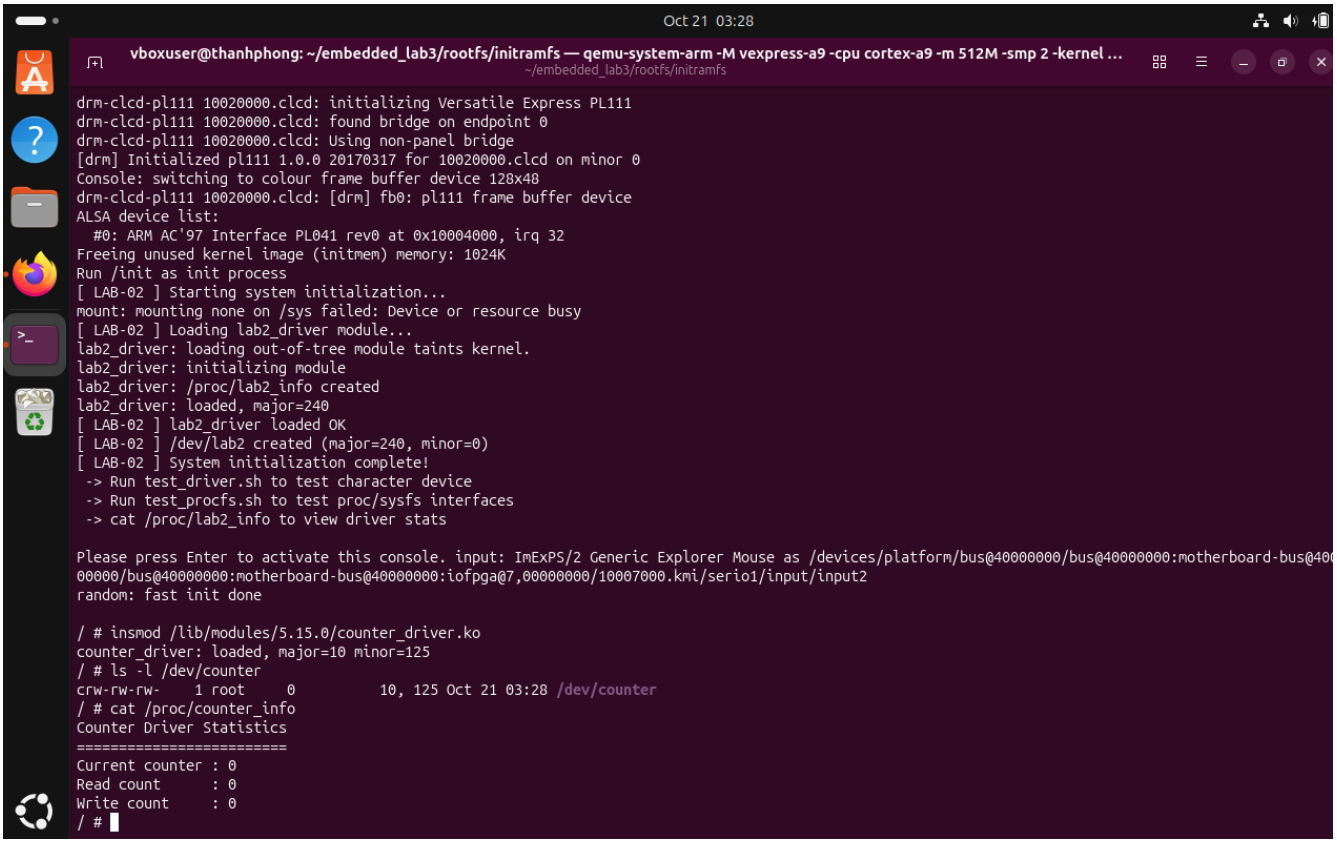


Figure 4: Task 2 - Device registration

Figure 4. Counter driver registration and initial statistics.

Figure 5. Three consecutive reads returning 0, 1, and 2.

Figure 6. Writing a valid integer and rejecting negative input.

Figure 7. Invalid input handling and module unloading.

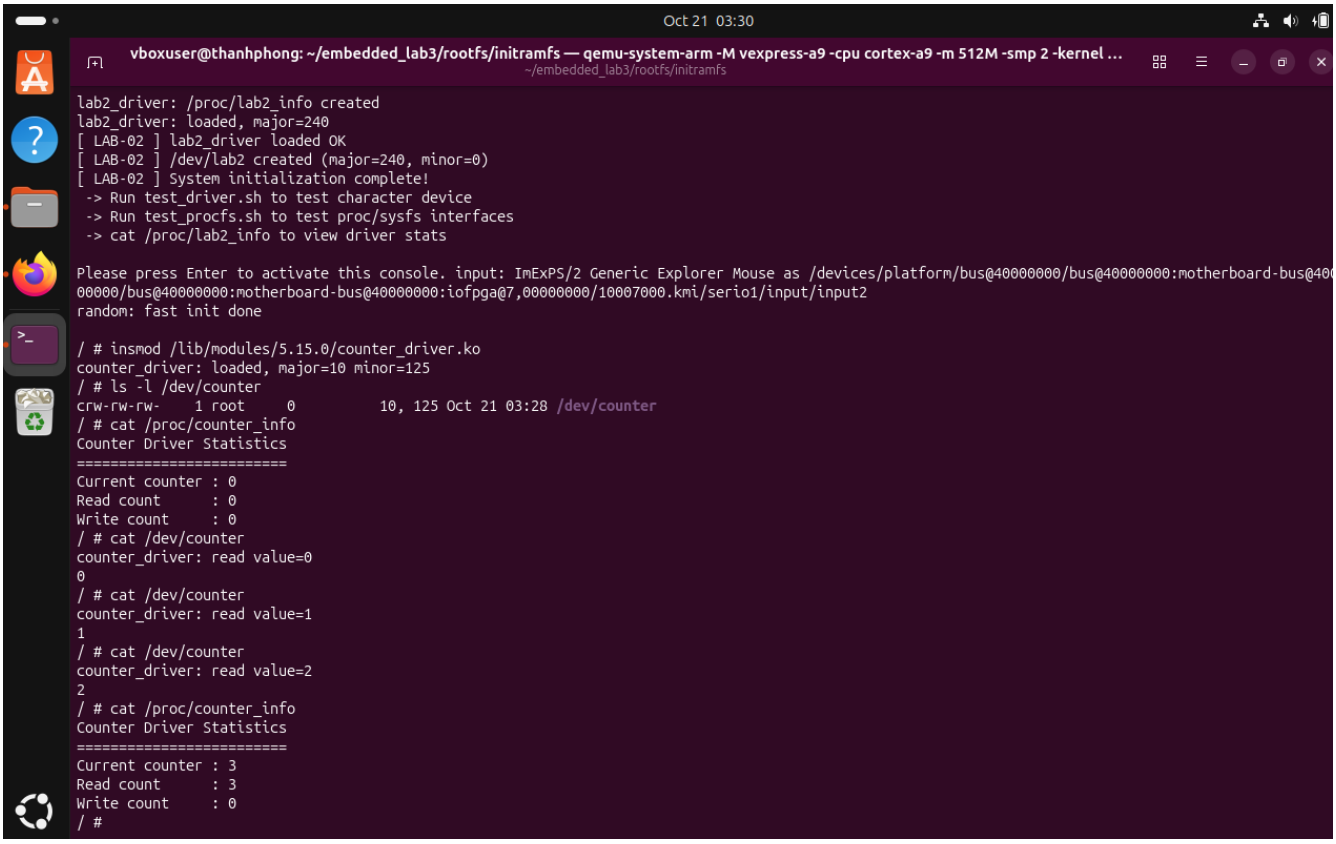


Figure 5: Task 2 - Counter reads

7. Task 4 – Theoretical Questions

Question 1: Why are module reference counts important?

The Linux kernel maintains module reference counts to track whether a kernel module is currently being used.

If the module is still in use, unloading it could remove functions or data that other kernel components are accessing.

The `THIS_MODULE` owner field in file_operations helps the kernel manage module references while a device is open.

This prevents unsafe removal of an actively used driver.

Question 2: Why must drivers use copy_to_user()?

Kernel space and user space have different memory access rules.

A pointer supplied by a user application cannot be treated as a trusted kernel pointer.

The `copy_to_user()` function safely transfers data from kernel memory to a user-space buffer and reports copying failures.

Similarly, `copy_from_user()` is used when receiving data from user space.

These functions help protect the kernel from invalid user-space memory addresses.

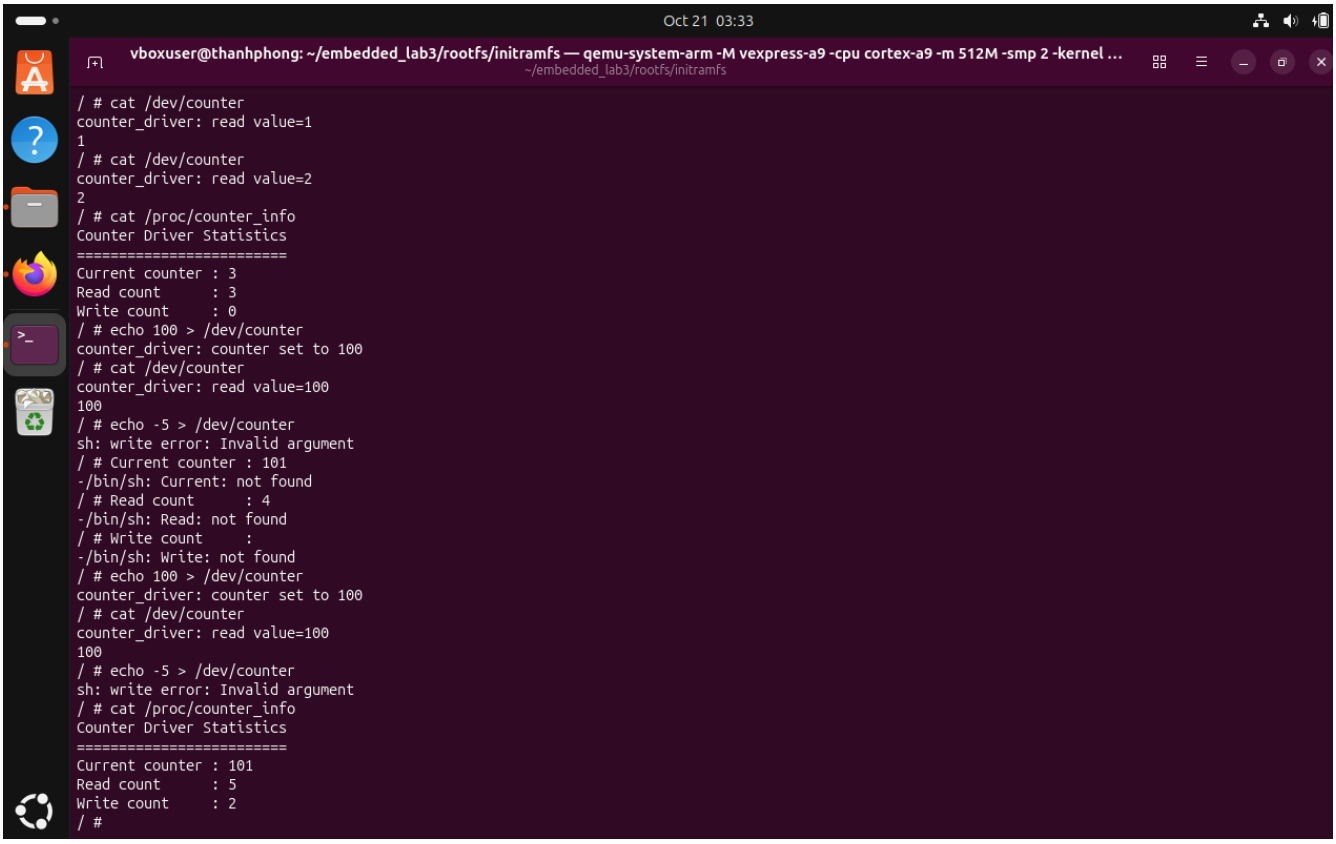


Figure 6: Task 2 - Counter writes

Question 3: MTD Devices vs Block Devices

MTD stands for Memory Technology Device.

MTD devices provide access to raw flash memory, such as NOR or NAND flash, and expose flash-specific operations.

Raw flash requires special handling for erase blocks, bad blocks, and wear.

Block devices present storage as numbered logical blocks and are commonly used with filesystems such as ext4.

Examples of block devices include SSDs and SD cards.

The main difference is that MTD exposes raw flash characteristics, while a block device provides a logical block-based storage interface.

Question 4: Why is the file offset checked in counter_read()?

The read() system call may be invoked repeatedly until the driver returns zero bytes.

Programs such as `cat` continue reading until they receive an end-of-file indication.

In the counter driver, `*offset > 0` causes subsequent reads on the same open file to return zero.

After the first successful read, the file offset is increased by the number of bytes returned.

Without this check, `cat /dev/counter` could repeatedly read and increment the counter without reaching EOF.

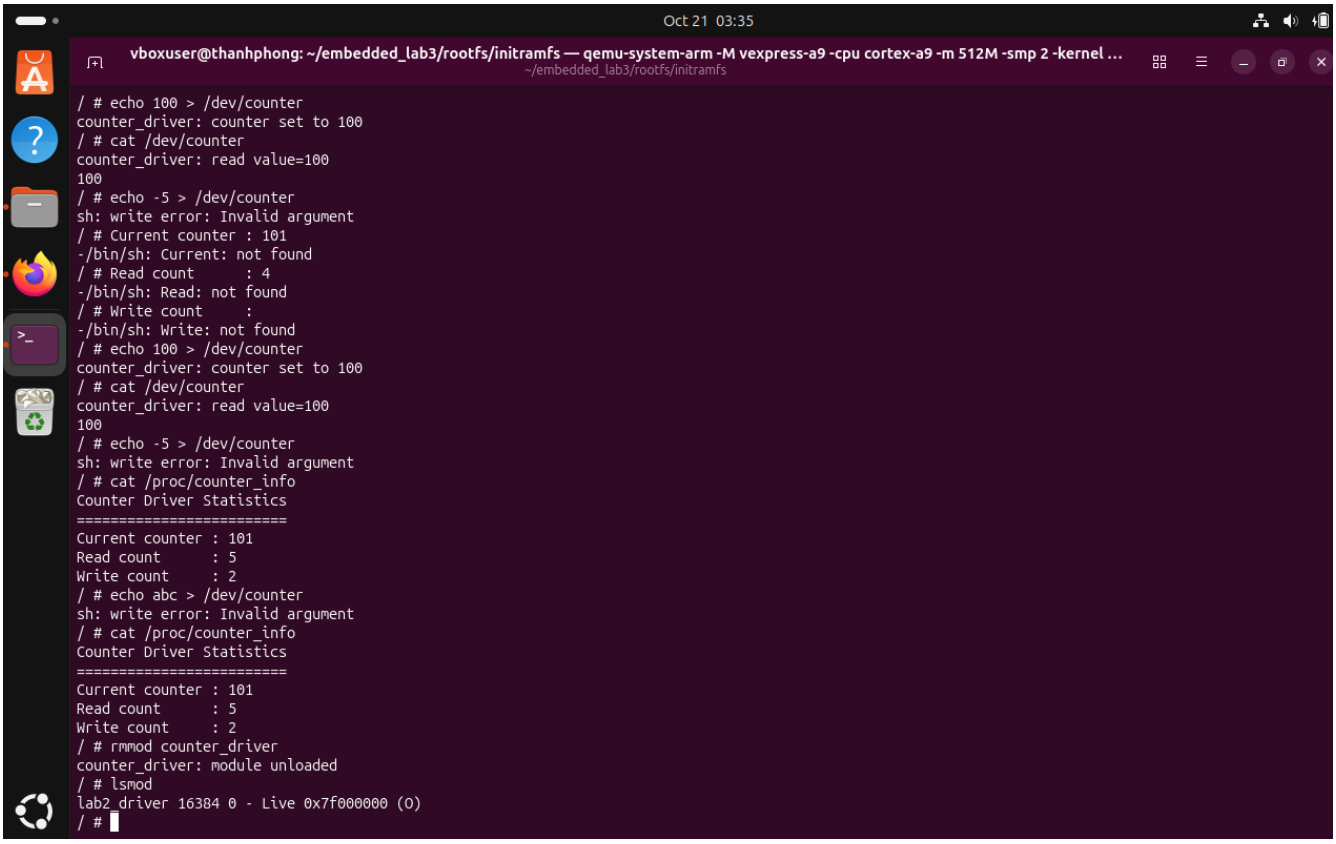


Figure 7: Task 2 - Invalid inputs

8. Conclusion

This assignment provided practical experience in developing and testing Linux kernel device drivers on an ARMv7 platform.

The first driver demonstrated character-device operations, ioctl-based control, shared-buffer management, and mutex synchronization.

The second driver demonstrated the miscdevice framework, atomic counter operations, input validation, and procfs statistics.

Both drivers were successfully compiled and tested using Linux kernel 5.15 under QEMU.

The experiments helped improve understanding of the interaction between user-space applications and Linux kernel modules.

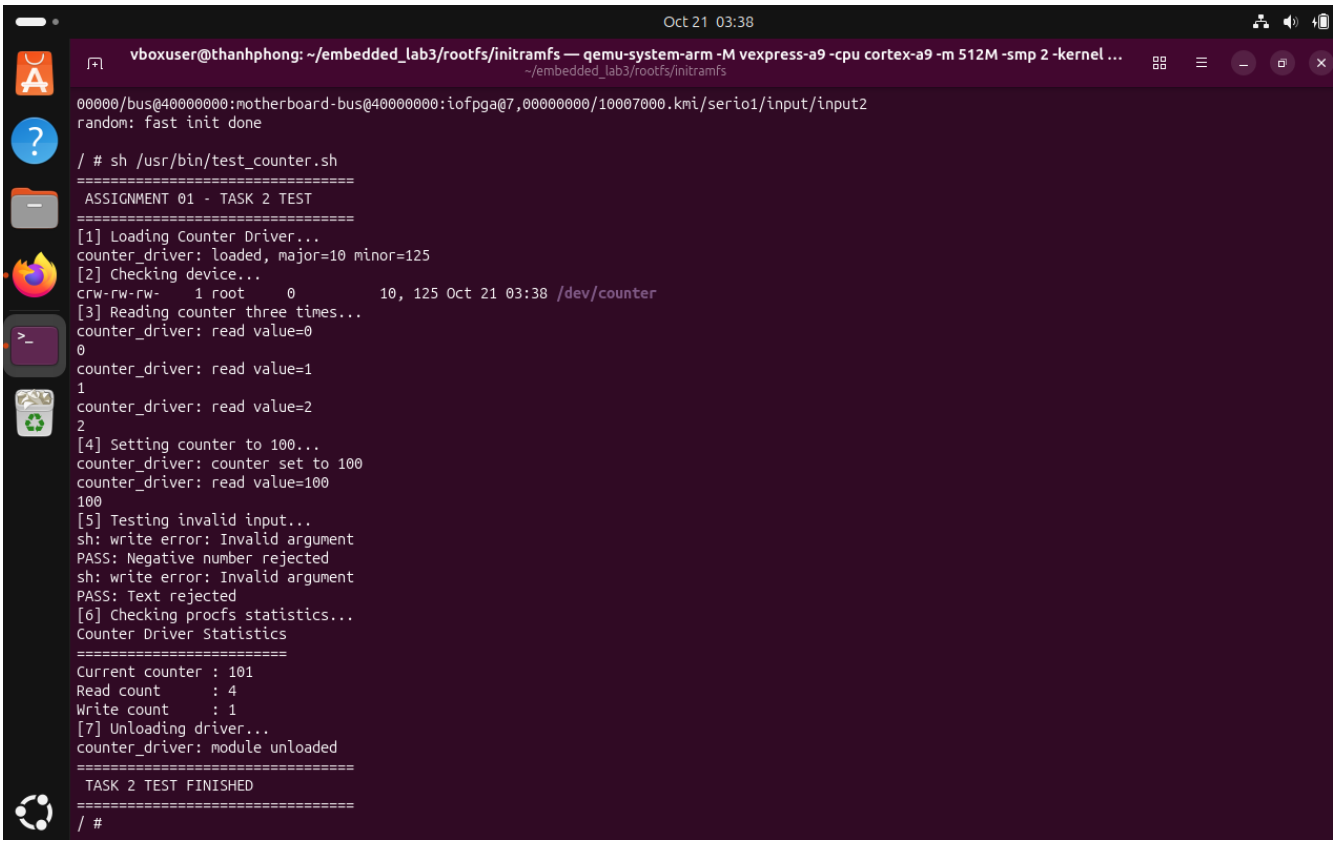


Figure 8: Task 2 - Automated testing